

```
In [7]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
In [8]: Airport = pd.read_excel("Airports.xlsx")
Airlines = pd.read_excel("Airlines.xlsx")
Runways = pd.read_excel("Runways.xlsx")
```

```
In [9]: Airport.head()
```

```
Out[9]:
```

	id	ident	type	name	latitude_deg	longitude_deg	elevation_ft	continent	iso_country	iso_region	municipality	scheduled_service	gps_code
0	6523	00A	heliport	Total Rf Heliport	40.070801	-74.933601	11.0	NaN	US	US-PA	Bensalem	no	00A
1	323361	00AA	small_airport	Aero B Ranch Airport	36.704022	-101.473911	3435.0	NaN	US	US-KS	Leoti	no	00AA
2	6524	00AK	small_airport	Lowell Field	59.947733	-151.692524	450.0	NaN	US	US-AK	Anchor Point	no	00AK
3	6525	00AL	small_airport	Epps Airpark	34.664799	-86.770302	820.0	NaN	US	US-AL	Harvest	no	00AL
4	6526	00AR	closed	Newport Hospital & Clinic Heliport	35.606700	-91.254896	237.0	NaN	US	US-AR	Newport	no	NaN

```
In [10]: Airport.shape
```

```
Out[10]: (73805, 18)
```

```
In [11]: Airlines.head()
```

```
Out[11]:
```

	id	airline	flight	airportfrom	airportto	dayofweek	time	length	delay
0	1	CO	269	SFO	IAH	3	15	205	1
1	2	US	1556	PHX	CLT	3	15	222	1
2	3	AA	2400	LAX	DFW	3	20	165	1
3	4	AA	2466	SFO	DFW	3	20	195	1
4	5	AS	108	ANC	SEA	3	30	202	0

```
In [5]: Airlines.head()
```

```
Out[10]: (73805, 18)
```

```
In [11]: Airlines.head()
```

```
Out[11]:
```

	id	airline	flight	airportfrom	airportto	dayofweek	time	length	delay
0	1	CO	269	SFO	IAH	3	15	205	1
1	2	US	1556	PHX	CLT	3	15	222	1
2	3	AA	2400	LAX	DFW	3	20	165	1
3	4	AA	2466	SFO	DFW	3	20	195	1
4	5	AS	108	ANC	SEA	3	30	202	0

```
In [5]: Airlines.head()
```

```
Out[5]:
```

	id	airline	flight	airportfrom	airportto	dayofweek	time	length	delay
0	1	CO	269	SFO	IAH	3	15	205	1
1	2	US	1556	PHX	CLT	3	15	222	1
2	3	AA	2400	LAX	DFW	3	20	165	1
3	4	AA	2466	SFO	DFW	3	20	195	1
4	5	AS	108	ANC	SEA	3	30	202	0

```
In [12]: Airlines.shape
```

```
Out[12]: (518556, 9)
```

```
In [13]: Runways.head()
```

```
Out[13]:
```

	id	airport_ref	airport_ident	length_ft	width_ft	surface	lighted	closed	iso_ident	iso_latitude_deg	iso_longitude_deg	iso_elevation_ft	iso_heading_deg_T
0	269408	6523	00A	80.0	80.0	ASPH-G	1	0	H1	NaN	NaN	NaN	NaN
1	256165	6524	00AK	2500.0	70.0	GRVL	0	0	N	NaN	NaN	NaN	NaN
2	254165	6525	00AL	2300.0	200.0	TURF	0	0	1	NaN	NaN	NaN	NaN
3	270502	6526	00AR	40.0	40.0	GRASS	0	0	H1	NaN	NaN	NaN	NaN
4	322128	322127	00AG	1480.0	60.0	Turf	0	0	1	NaN	NaN	NaN	NaN

```
In [14]: Runways.shape
```

```
Out[14]: (43977, 28)
```

```
In [15]: Runways.info()
```

```

In [13]: runways.head()

Out[13]:
   id  airport_ref  airport_ident  length_ft  width_ft  surface  lighted  closed  is_ident  is_latitude_deg  is_longitude_deg  is_elevation_ft  is_heading_degT
0  269408         6523          00A         80.0       80.0  ASPH+G         1         0         H1             NaN             NaN             NaN             NaN
1  255155         6524          00AK        2500.0       70.0   GRVL         0         0         N             NaN             NaN             NaN             NaN
2  254165         6525          00AL        2000.0      200.0   TURF         0         0         1             NaN             NaN             NaN             NaN
3  270932         6526          00AR         40.0       40.0  GRASS         0         0         H1             NaN             NaN             NaN             NaN
4  322128        322127          00AS        1450.0       80.0   Turf         0         0         1             NaN             NaN             NaN             NaN

In [14]: Runways.shape
Out[14]: (43977, 20)

In [15]: Runways.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 43977 entries, 0 to 43976
Data columns (total 20 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   id                   43977 non-null  int64
1   airport_ref          43977 non-null  int64
2   airport_ident        43977 non-null  object
3   length_ft            43753 non-null  float64
4   width_ft             41888 non-null  float64
5   surface              43528 non-null  object
6   lighted              43977 non-null  int64
7   closed               43977 non-null  int64
8   is_ident              43793 non-null  object
9   is_latitude_deg      15816 non-null  float64
10  is_longitude_deg      15880 non-null  float64
11  is_elevation_ft       12781 non-null  float64
12  is_heading_degT       14624 non-null  float64
13  is_displaced_threshold_ft  2883 non-null  float64
14  he_ident              37332 non-null  object
15  he_latitude_deg       14971 non-null  float64
16  he_longitude_deg      14973 non-null  float64
17  he_elevation_ft       12628 non-null  float64
18  he_heading_degT       16428 non-null  float64
19  he_displaced_threshold_ft  3176 non-null  float64
dtypes: float64(12), int64(4), object(4)
memory usage: 6.7+ MB

In [9]: Runways.drop(['is_ident', 'is_latitude_deg', 'is_longitude_deg', 'is_elevation_ft', 'is_heading_degT', 'is_displaced_threshold_ft',
                    'he_displaced_threshold_ft'], axis = 1, inplace=True)

In [16]: Runways

```

```

In [16]: Runways
Out[16]:
   id  airport_ref  airport_ident  length_ft  width_ft  surface  lighted  closed  is_ident  is_latitude_deg  is_longitude_deg  is_elevation_ft  is_heading_degT
0  269408         6523          00A         80.0       80.0  ASPH+G         1         0         H1             NaN             NaN             NaN             NaN
1  255155         6524          00AK        2500.0       70.0   GRVL         0         0         N             NaN             NaN             NaN             NaN
2  254165         6525          00AL        2000.0      200.0   TURF         0         0         1             NaN             NaN             NaN             NaN
3  270932         6526          00AR         40.0       40.0  GRASS         0         0         H1             NaN             NaN             NaN             NaN
4  322128        322127          00AS        1450.0       80.0   Turf         0         0         1             NaN             NaN             NaN             NaN
...
43972  225185         27243          ZYTX        10495.0      148.0   CON         1         0         E         41.830402        123.489002        171.0             4
43973  225189         27244          ZYJY         8530.0      148.0   CON         1         0         S         42.881100        129.436005        623.0             8
43974  364567        317861          ZYVC         8000.0       NaN       NaN         0         0         4             NaN             NaN             NaN             NaN
43975  340709        346706          ZZ-3000        1900.0       15.0   Turf         0         0         15            NaN             NaN             NaN             NaN
43976  313863        313626          ZZZZ         1713.0       82.0  concrete         0         0         16         30.783501        130.272995             NaN             N

43977 rows x 20 columns

In [17]: Airport.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 73885 entries, 0 to 73884
Data columns (total 18 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   id                   73885 non-null  int64
1   ident               73885 non-null  object
2   type                73885 non-null  object
3   name                 73885 non-null  object
4   latitude_deg         73885 non-null  float64
5   longitude_deg         73885 non-null  float64
6   elevation_ft         59983 non-null  float64
7   continent            38886 non-null  object
8   iso_country          73246 non-null  object
9   iso_region           73885 non-null  object
10  municipality         68739 non-null  object
11  scheduled_service     73885 non-null  object
12  gps_code             42996 non-null  object
13  iata_code            9160 non-null   object
14  local_code           32975 non-null  object
15  home_link            3492 non-null   object
16  wikipedia_link        18785 non-null  object
17  keywords             13751 non-null  object
dtypes: float64(3), int64(1), object(14)
memory usage: 16.1+ MB

In [12]: Airport.drop(['continent', 'iso_country', 'iso_region', 'municipality', 'gps_code', 'local_code', 'home_link',
                    'wikipedia_link', 'keywords'], axis=1, inplace=True)

Airport

```

```
Out[12]:
```

	id	ident	type	name	latitude_deg	longitude_deg	elevation_ft	scheduled_service	iata_code
0	8820	00A	heliport	Total F4 Heliport	40.070801	-74.033601	11.0	no	NaN
1	320361	00AA	small_airport	Aure B Ranch Airport	38.704022	-101.475611	9456.0	no	NaN
2	8804	00AK	small_airport	Lowell Field	60.647723	-151.802851	450.0	no	NaN
3	8806	00AL	small_airport	Epps Airpark	34.864769	-88.770302	800.0	no	NaN
4	8808	00AR	closed	Newport Hospital & Clinic Heliport	35.608700	-91.254888	237.0	no	NaN
...	...	...	...	...	...	...	...	...	...
73800	48376	ZZ0001	heliport	Sealand Heliport	51.834444	1.485500	40.0	no	NaN
73801	30726	ZZ0002	small_airport	Gorree Islands Airbase	-11.594278	47.796369	11.0	no	NaN
73802	346786	ZZ0003	small_airport	Fairling Gost Airport	32.116567	-97.356312	690.0	no	NaN
73803	342102	ZZZW	closed	Scandinium City Heliport	69.358267	-130.636310	4.0	no	ZYW
73804	310326	ZZZZ	small_airport	Satsuna Hgima Airport	30.784722	130.270556	308.0	no	NaN

73805 rows x 9 columns

```
In [18]: Airport_runway = pd.merge(Airport_runways, left_on = "ident", right_on = "airport_ident")
```

```
In [19]: Airport_runway.drop(['id_x', 'id_y'], axis = 1, inplace=True)
```

```
In [20]: Airport_runway
```

```
Out[20]:
```

	ident	type	name	latitude_deg	longitude_deg	elevation_ft	continent	iso_country	iso_region	municipality	...	is_longitude_deg	is
0	00A	heliport	Total F4 Heliport	40.070801	-74.033601	11.0	NaN	US	US-PA	Berksham	...	NaN	NaN
1	00AA	small_airport	Lowell Field	60.647733	-151.802804	450.0	NaN	US	US-AK	Anchor Point	...	NaN	NaN
2	00AL	small_airport	Epps Airpark	34.864796	-88.770302	800.0	NaN	US	US-AL	Harvest	...	NaN	NaN
3	00AR	closed	Hospital & Clinic Heliport	35.608700	-91.254888	237.0	NaN	US	US-AR	Newport	...	NaN	NaN
4	00AS	small_airport	Fulton Airport	34.942803	-97.518019	1100.0	NaN	US	US-OK	Alex	...	NaN	NaN
...	...	...	...	...	...	...	...	...	...	...	...	...	...
43973	ZYTX	large_airport	Shenyang Taishan International Airport	41.830801	123.480302	106.0	AS	CN	CN-21	Human Shenyang	...	123.480302	NaN
43974	ZYUJ	medium_airport	Yanji Chaoyangchuan Airport	42.850801	129.451004	624.0	AS	CN	CN-22	Yanji	...	129.450005	NaN
43975	ZYXX	medium_airport	Yingkou Langji Airport	40.542204	122.300000	NaN	AS	CN	CN-21	Liaohian Yingkou	...	NaN	NaN
43976	ZZ0001	small_airport	Fairling Gost Airport	32.110337	-97.350312	690.0	NaN	US	US-TX	Blair	...	NaN	NaN
43977	ZZZZ	small_airport	Satsuna Hgima Airport	30.784722	130.270556	308.0	AS	JP	JP-46	Moshira	...	130.272965	NaN

43977 rows x 13 columns

```
In [44]: final_df = pd.merge(Airlines, Airport_runways, how = 'inner', left_on = 'airport_from', right_on = 'iata_code')
```

```
In [25]: final_df.drop_duplicates(subset=['id'], keep='first', inplace=True)
```

```
Out[25]:
```

	id	Airline	Flight	AirportFrom	AirportTo	DayOfWeek	Time	Length	Delay	ident	is_longitude_deg	is_elevation_ft	is_heading_degT	is_id
0	1	CO	269	SFO	LAX	3	15	215	1	KSFO	-122.362004	100	28.8	NaN
4	4	AA	2496	SFO	DFW	3	20	195	1	KSFO	-122.362004	100	28.8	NaN
6	8	DL	2606	SFO	MSP	3	35	218	1	KSFO	-122.362004	100	28.8	NaN
12	129	DL	1680	SFO	DTW	3	545	270	0	KSFO	-122.362004	100	28.8	NaN
16	160	UA	756	SFO	CDN	3	548	168	0	KSFO	-122.362004	100	28.8	NaN
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2160216	451044	CO	2	GUM	HNL	1	400	430	1	PGUM	144.763997	2390	65.8	NaN
2160216	469900	CO	2	GUM	HNL	2	400	430	1	PGUM	144.763997	2390	65.8	NaN
2160216	480300	CO	2	GUM	HNL	3	400	430	0	PGUM	144.763997	2390	65.8	NaN
2160217	608856	CO	2	GUM	HNL	4	400	430	1	PGUM	144.763997	2390	65.8	NaN
2160214	528138	CO	2	GUM	HNL	5	400	430	1	PGUM	144.763997	2390	65.8	NaN

518025 rows x 14 columns

```
In [26]: url = "https://en.wikipedia.org/wiki/List_of_airlines_of_the_United_States"
tables = pd.read_html(url)
```

```
In [27]: print(tables)
```

```
[
  Airline
  Image
  TATA
  TCAO
  Callign
  \
0
Alaska Airlines
NaN
AS
ASA
ALASKA
1
Allegiant Air
NaN
G4
AAY
ALLEGANT
2
American Airlines
NaN
AA
AAL
AMERICAN
3
Avelo Airlines
NaN
XP
VOP
AVELO
4
Breeze Airways
NaN
MQ
MOY
BREEZE
5
Delta Air Lines
NaN
DL
DAL
DELTA
6
Frontier Airlines
NaN
F3
FFT
FRONTIER FLIGHT
7
Hawaiian Airlines
NaN
HA
HAL
HAWAIIAN
8
JetBlue
NaN
B6
JBU
JETBLUE
9
Southwest Airlines
NaN
WN
SWA
SOUTHWEST
10
Spirit Airlines
NaN
NK
NKS
SPIRIT WINGS
11
Sun Country Airlines
NaN
SY
SCX
SUN COUNTRY
12
United Airlines
NaN
UA
UAL
UNITED

Primary hubs, secondary hubs
Founded
\
0
Seattle/Tacoma Anchorage Portland (OR) San Diego...
1932
1
Las Vegas Cincinnati Destin Ft. Walton Beach T...
1997
2
Dallas/Fort Worth Charlotte Miami Chicago-O'Hare...
1926
3
Phoenix Sky Harbor International Airport
1935
]
```

```
In [28]: tables[0]
```

In [20]: tables[6]

Out[20]:

	Airline	Image	IATA	ICAO	Call sign	Primary hubs, secondary hubs	Founded	Note
0	Comco	NaN	NaN	NaN	NaN	NaN	2002	NaN
1	Jared	NaN	NaN	VWW	JANET	Las Vegas	1972	NaN
2	Justice Prisoner and Alien Transportation System	NaN	NaN	JUD	JUSTICE	Oklahoma City	1900	Commenced operations in 1905.

In [30]: # Let's first merge all wikipedia tables.
wiki_table = [tables[0], tables[1], tables[2], tables[3], tables[4], tables[5], tables[6]]

In [31]: wiki_tables = pd.concat(wiki_table, ignore_index=True)

In [33]: wiki_tables

Out[33]:

	Airline	Image	IATA	ICAO	Call sign	Primary hubs, secondary hubs	Founded	Note
0	Alaska Airlines	NaN	AS	ASA	ALASKA	Seattle-Tacoma Anchorage-Denver (OR) San Diego	1932.0	Founded as McGee Airways and commenced operations in 1932.
1	Allegiant Air	NaN	G4	AVY	ALLEGiant	Las Vegas Cincinnati Destin Ft. Walton Beach L.	1997.0	Founded as WestJet Express and began operations in 1997.
2	American Airlines	NaN	AA	AAL	AMERICAN	Dallas/Fort Worth Charlotte Miami Chicago-O'Hare	1926.0	Founded as American Airways and commenced operations in 1926.
3	Avelo Airlines	NaN	XP	VOP	AUCLLO	Subsidiary of Hawaiian Airlines	1987.0	First, it did business as Casino Express Airline.
4	Breeze Airways	NaN	MX	MOY	MOXY	Charlotte (NC) Hartford New Orleans Norfolk P.	2018.0	Founded as Many Airways, but was renamed due to legal issues.
...	...	...	...	...	...	...	...	...
127	Liftair	NaN	NaN	NaN	NaN	NaN	NaN	NaN
128	Life Line	NaN	NaN	NaN	NaN	NaN	NaN	NaN
129	Comco	NaN	NaN	NaN	NaN	NaN	2002.0	NaN
140	Jared	NaN	NaN	VWW	JANET	Las Vegas	1972.0	NaN
141	Justice Prisoner and Alien Transportation System	NaN	NaN	JUD	JUSTICE	Oklahoma City	1900.0	Commenced operations in 1905.

142 rows x 9 columns

In [34]: wiki_df = wiki_tables[['IATA', 'Founded']]

Out[34]:

	IATA	Founded
0	AS	1932.0
1	G4	1997.0
2	AA	1926.0
3	XP	1987.0
4	MX	2018.0
...	...	...
127	NaN	NaN
128	NaN	NaN
129	NaN	2002.0
140	NaN	1972.0
141	NaN	1900.0

142 rows x 2 columns

In [35]: # Now we gather all the information that we got from wikipedia link and the data that we have.
df = final_df.merge(wiki_df, left_on='Airline', right_on='IATA')

In [36]: df

Out[36]:

	id	Airline	Flight	Airport from	Airport to	Day/Off week	Time	Length	Delay	ident	is_heading_degI	is_displaced_threshold	is_heading_degE
0	4	AA	2446	SFO	DFW	3	20	195	1	KSFO	...	28.0	491.0
1	221	AA	526	SFO	DFW	3	360	215	0	KSFO	...	28.0	491.0
2	234	AA	552	SFO	MDA	3	360	315	1	KSFO	...	28.0	491.0
3	905	AA	610	SFO	ORD	3	365	255	0	KSFO	...	28.0	491.0
4	1739	AA	74	SFO	JFK	3	425	325	1	KSFO	...	28.0	491.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
434915	491838	GC	4292	LVR	JFK	3	360	110	1	KLVR	...	35.3	NaN
434930	516333	GC	4292	LVR	JFK	4	360	110	0	KLVR	...	35.3	NaN
434931	834123	GC	4292	LVR	JFK	5	360	115	0	KLVR	...	35.3	NaN
434932	690568	GC	3752	ABR	MSP	7	410	78	1	KABR	...	135.1	NaN
434933	180396	GC	3752	ABR	MSP	7	410	76	0	KABR	...	135.1	NaN

434924 rows x 14 columns

In [37]: # Now let's use the web scraping to import the data from the wikipedia.

```
url12 = "https://en.wikipedia.org/wiki/List_of_the busiest airports in the United States"  
tables = pd.read_html(url12)
```

[illegible]

434824 rows x 47 columns

```
In [37]: # Now Lets use the web scrapping to import the data from the wikipedia.

ur12 = "https://en.wikipedia.org/wiki/List_of_the_nuclear_airports_in_the_United_States"

tables = pd.read_html(ur12)
```

In [38]:	tables
----------	--------

```
Out[38]:
```

	0	1
0	Mus This graph was using the Legacy Graph and Semantics... (Large) IATA Code	
1	June (2024)	
2	Hartsfield-Jackson Atlanta International Airport	ATL
3	Dallas Fort Worth International Airport	DFW
4	Denver International Airport	DEN
5	O'Hare International Airport	ORD
6	Los Angeles International Airport	LAX
7	John F. Kennedy International Airport	JFK
8	Charlotte Douglas International Airport	CLT
9	Narry Reid International Airport	LAS
10	Orlando International Airport	MCO
11	Miami International Airport	MIA
12	Phoenix Sky Harbor International Airport	PHX
13	Seattle-Tacoma International Airport	SEA
14	San Francisco International Airport	SFO
15	New Liberty International Airport	EWR
16	George Bush Intercontinental Airport	IAH
17	Logan International Airport	BOS

```
In [39]: tables[1] = tables[1].drop(['2024[2]', '2021[5]', '2020[6]'], axis=1, errors='ignore')
```

```
In [40]: tables[1].head()
```

Out[48]:	Major US Airports by Passenger Volume (2018)										
	Rank (2018)	Airport (large)	IATA Code	Major Cities Served	Metropolitan Area	State	2018Q1	2018Q2	2018Q3	2018Q4	Annual Total
0	1	Hartsfield-Jackson Atlanta International Airport	ATL	Atlanta	Metropolitan Atlanta	GA	5,069,008	4,500,001	5,305,705	5,186,777	5,425,084
1	2	Dallas Fort Worth International Airport	DFW	Dallas and Fort Worth	Dallas-Fort Worth	TX	3,424,412	3,545,138	3,673,873	3,281,774	3,186,623
2	3	Denver International Airport	DEN	Denver	Greater Denver	CO	3,780,187	3,077,332	3,350,245	3,142,611	2,600,007

```
In [41]: print(tables[2].columns)
```

```
Index(['Rank (2023)', 'Airports (medium hubs)', 'IATA Code', 'City served',  
      'Metro Area', 'State', '2024[2]', '2023[3]', '2022[4]', '2021[5]',  
      '2020[6]', '2019[7]', '2018[8]', '2017[9]', '2016[10]', '2015[11]'],  
      dtype='object')
```

```
In [42]: print(tables[3].columns)
```

```
Index(['Rank', 'Airport name', 'Location', 'IATA Code', '2024[12]', '2023[13]',  
      '2022[14]', '2021[15]', '2020[16]', '2019[17]'],  
      dtype='object')
```

```
In [43]: print(tables[4].columns)
```

```
MultiIndex([( 'Rank', 'Rank'),
            ('Airport name', 'Airport name'),
            ('Location', 'Location'),
            ('IATA code', 'IATA code'),
            ('Cargo', 'Ibs.'),
            ('Cargo', '% chg. 2023/2024')],
```

```
In [44]: print(tables[5].columns)
```

```
Index[["mw-parser-output .nav{display:inline;font-size:88%;font-weight:normal;mw-parser-output .navbar-collapse{float:left;text-align:left;mw-parser-output .navbar-boxtext{word-spacing:0;}mw-parser-output .navbar ul{display:inline-block;white-space:nowrap;line-height:inherit;}mw-parser-output .navbar-brackets:before{margin-left:-0.125em;content:""}mw-parser-output .navbar-brackets:after{margin-left:-0.125em;content:""}mw-parser-output .navbar li{word-spacing:-0.125em}mw-parser-output .navabar aspan,mw-parser-output .navabar abbr{text-decoration:inherit;}mw-parser-output .navbar-ninl abbr{font-variant:small-caps;border-bottom:none;text-decoration:none;color:inherit;}mw-parser-output .navbar-ct-full{font-size:114%;margin:7em}mw-parser-output .navbar-ct-min{font-size:114%;margin:0 4em}.html.skin-theme-clientpref-night .mw-parser-output .navbar li a abbr{color:var(--color-base)!important}@media(prefers-color:scheme:dark){.html.skin-theme-clientpref-os .mw-parser-output .navbar li a a abbr{color:var(--color-base)!important}@media print{.mw-parser-output .navbar{display:none!important}}}]&#x2D;for airports in the United States", "mw-parser-output .navbar{display:inline;font-size:88%;font-weight:normal;mw-parser-output .navbar-collapse{float:left;text-align:left;mw-parser-output .navbar-boxtext{word-spacing:0;}mw-parser-output .navbar ul{display:inline-block;white-space:nowrap;line-height:inherit;}mw-parser-output .navbar-brackets:before{margin-left:-0.125em;content:""}mw-parser-output .navbar-brackets:after{margin-left:-0.125em;content:""}mw-parser-output .navbar li{word-spacing:-0.125em}mw-parser-output .navabar aspan,mw-parser-output .navabar abbr{text-decoration:inherit;}mw-parser-output .navbar-ninl abbr{font-variant:small-caps;border-bottom:none;text-decoration:none;color:inherit;}mw-parser-output .navbar-ct-full{font-size:114%;margin:7em}
```



```
In [49]: tables[1][ 'traffic_chg18_19' ] = tables[1][ 2018[0] ] - tables[1][ 2019[1] ]

In [49]: tables[1][ 'traffic_chg17_18' ] = tables[1][ '2017[9]' ] - tables[1][ '2018[10]' ]
tables[1][ 'hubs' ] = str('large_hub')

In [50]: tables[1] = tables[1][ ['IATA Code', 'traffic_chg18_19', 'traffic_chg17_18', 'hubs' ] ]
tables[2]

-----
KeyError                                Traceback (most recent call last)
/tmp/ipykernel_68/3886136134.py in <cell line: 1>()
----> 1 tables[1] = tables[1][ ['IATA Code', 'traffic_chg18_19', 'traffic_chg17_18', 'hubs' ] ]
      2 tables[2]

/usr/local/lib/python3.10/site-packages/pandas/core/frame.py in _getitem_(self, key)
   3811     if is_iterator(key):
   3812         key = list(key)
-> 3813     indexer = self.columns._get_indexer_strict(key, "columns")[1]
   3814
   3815     # take() does not accept boolean indexes

/usr/local/lib/python3.10/site-packages/pandas/core/indexes/base.py in _get_indexer_strict(self, key, axis_name)
   6608     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
   6609
-> 6670     self._raise_if_missing(keyarr, indexer, axis_name)
   6671
   6672     keyarr = self.take(indexer)

/usr/local/lib/python3.10/site-packages/pandas/core/indexes/base.py in _raise_if_missing(self, key, indexer, axis_name)
   6121
   6122     not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique())
-> 6133     raise KeyError(f"{not_found} not in index")
   6134
   6135     @overload
KeyError: ["traffic_chg18_19", 'traffic_chg17_18', 'hubs'] not in index"
```

```
In [51]: print(tables[1].head())

Rank (2024)                                Airports (Large) IATA Code \
0      1      Hartsfield-Jackson Atlanta International Airport    ATL
1      2      Dallas Fort Worth International Airport            DFW
2      3      Denver International Airport                       DEN
3      4      O'Hare International Airport                       ORD
4      5      Los Angeles International Airport                  LAX

Major cities served      Metro area State  2023[3]  2023[4] \
0      Atlanta          Metro Atlanta    GA  50950008  45790001
1      Dallas Fort Worth Dallas-Fort Worth TX  39360232  35345138
2      Denver            Greater Denver    CO  37863907  33773832
3      Chicago            Chicagoland     IL  35843184  33128474
4      Los Angeles        Greater Los Angeles CA  40956673  32226616

2019[7]  2018[8]  2017[9]  2016[10]  2015[11]  traffic_chg18_19 \
0  51505795  51865797  50251964  50501858  49348732  -1639998
1  31778573  32824799  31816933  31283579  31589839  -2956774
2  31592943  31362941  29889957  28207294  26388943  -2238894
3  44871222  36873017  38503908  37589809  36385658  -907206
4  41533184  42624058  41232432  39630842  36331272  -315054

traffic_chg17_18      hubs
0      -160904      large hub
```

```
In [52]: tables[1] = tables[1][ ['IATA Code', 'traffic_chg18_19', 'traffic_chg17_18', 'Airports' ] ]
tables[1]

-----
KeyError                                Traceback (most recent call last)
/tmp/ipykernel_68/286126398.py in <cell line: 1>()
----> 1 tables[1] = tables[1][ ['IATA Code', 'traffic_chg18_19', 'traffic_chg17_18', 'Airports' ] ]
      2 tables[1]

/usr/local/lib/python3.10/site-packages/pandas/core/frame.py in _getitem_(self, key)
   3813     if is_iterator(key):
   3814         key = list(key)
-> 3815     indexer = self.columns._get_indexer_strict(key, "columns")[1]
   3816
   3817     # take() does not accept boolean indexes

/usr/local/lib/python3.10/site-packages/pandas/core/indexes/base.py in _get_indexer_strict(self, key, axis_name)
   6608     keyarr, indexer, new_indexer = self._reindex_non_unique(keyarr)
   6609
-> 6670     self._raise_if_missing(keyarr, indexer, axis_name)
   6671
   6672     keyarr = self.take(indexer)

/usr/local/lib/python3.10/site-packages/pandas/core/indexes/base.py in _raise_if_missing(self, key, indexer, axis_name)
   6133
   6134     not_found = list(ensure_index(key)[missing_mask.nonzero()[0]].unique())
-> 6135     raise KeyError(f"{not_found} not in index")
   6136
   6137     @overload
KeyError: ["Airports"] not in index"
```

```
In [53]: # I use first merge all wikipedia table.
wiki_data = [ tables[1], tables[2] ]
wiki_data = pd.concat(wiki_data, ignore_index=True)
wiki_data
```

```
Out[53]:
```

Rank (2024)	Airports (Big)	IATA Code	Major cities served	Metro area	State	2015[3]	2022[4]	2019[7]	2018[8]	...	traffic_chg18_19	traffic_chg17_18	hubs	Rank (2024)
0	1.0	ATL	Atlanta	Metro Atlanta	GA	50650088	45388001	51605705	51884797	...	-1636020.6	240894.0	large_hub	NaN
1	2.0	DFW	Dallas and Fort Worth	Dallas-Fort Worth	TX	38548212	36345118	35778573	32824798	...	-2956774.0	531354.0	large_hub	NaN
2	3.0	International	Denver	Greater Denver	CO	37863907	33773832	33600445	31582041	...	-2238804.0	-161705.0	large_hub	NaN

83 rows x 23 columns

```
In [55]: final_df
```

Out[55]:

Out[55]:

240003 — x 78 —

358002 rows x 70 columns

```

class 'pandas.core.frame.DataFrame'
Int64Index: 354081 entries, 0 to 354081
Data columns (total 79 columns):
 #   column                                     Non-Null Count  Dtype  
---  -
 0   id                                         354082 non-null  int64   
 1   Airline                                   354082 non-null  object  
 2   Flight                                    354082 non-null  int64   
 3   TailNum                                  354082 non-null  object  
 4   AirportTo                                 354082 non-null  object  
 5   DayOfWeek                                354082 non-null  int64   
 6   Time                                     354082 non-null  int64   
 7   Length                                   354082 non-null  object  
 8   Delay                                     354082 non-null  int64   
 9   Ident                                     354082 non-null  object  
10   type                                      354082 non-null  object  
11   latitude_deg                             354082 non-null  float64  
12   longitude_deg                            354082 non-null  float64  
13   elevation_ft                             354082 non-null  float64  
14   continent                                 5807 non-null    object  
15   iso_country                              354082 non-null  object  
16   iso_region                              354082 non-null  object  
17   scheduled_service                       354082 non-null  object  
18   gps_code                                 354082 non-null  object  
19   iata_code                                354082 non-null  object  
20   icao_code                                354082 non-null  object  
21   home_link                                293954 non-null  object  
22   wikipedia_link                           35555 non-null   object  
23   keywords                                  21971 non-null   object  
24   wikipedia_ref                             2002 non-null    object  
27   airport_ident                             354082 non-null  object  
28   length_ft                                 354082 non-null  float64  
29   width_ft                                 354082 non-null  float64  
30   height_ft                                 354082 non-null  float64  
31   lighted                                   354082 non-null  int64   
32   closed                                    354082 non-null  int64   
33   is_land                                    354082 non-null  int64   
34   is_water                                  354082 non-null  int64   
35   is_longitude_deg                         354082 non-null  float64  
36   is_elevation_ft                          354082 non-null  float64  
37   is_heading_deg                            354082 non-null  float64  
38   is_displaced_threshold_ft                82 non-null     float64  
39   is_ident                                   354082 non-null  object  
40   is_latitude_deg                          354082 non-null  float64  
41   is_longitude_deg                         354082 non-null  float64  
42   is_elevation_ft                          355183 non-null  float64  
43   is_heading_deg                            354082 non-null  float64  
44   is_displaced_threshold_ft                57764 non-null   float64  
45   founded                                  354082 non-null  float64  
46   founded                                  354082 non-null  float64  
47   Nats (2024)                             274167 non-null  object  
48   Airports (large)                         274167 non-null  object  
49   Airports (small)                         274167 non-null  object  
50   Major cities served                     274167 non-null  object

```

```
dtypes: float64(23), int64(16), object(31)
memory usage: 193.9+ MB
```

```
In [47]: final_df.drop(['ic', 'AirportFrom', 'airport_id', 'iata_code', 'AirportTo', 'surface', 'ident', 'IATA', 'IATA Code', 'name'], axis=1,
```

```
In [50]: final_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 358082 entries, 0 to 358081
Data columns (total 69 columns):
 #   Column                                Non-Null Count  Dtype
---  --
 0   Airline                             358082 non-null object
 1   Flight                             358082 non-null int64
 2   DayOfWeek                           358082 non-null int64
 3   Time                               358082 non-null int64
 4   Length                             358082 non-null int64
 5   Delay                             358082 non-null int64
 6   Type                               358082 non-null object
 7   latitude_deg                       358082 non-null float64
 8   longitude_deg                     358082 non-null float64
 9   elevation_ft                      358082 non-null float64
10   continent                          5827 non-null object
11   iso_country                        358082 non-null object
12   iso_region                        358082 non-null object
13   municipality                      358082 non-null object
14   scheduled_service                 358082 non-null object
15   gps_code                         358082 non-null object
16   local_code                       358082 non-null object
17   home_link                         294994 non-null object
18   wikipedia_link                   155516 non-null object
19   keywords                          211653 non-null object
20   airport_ref                       358082 non-null int64
21   length_ft                         358082 non-null float64
22   width_ft                         358082 non-null float64
23   lighted                          358082 non-null int64
24   closed                           358082 non-null int64
25   le_ident                          358082 non-null object
26   le_latitude_deg                   358082 non-null float64
27   le_longitude_deg                  358082 non-null float64
28   le_elevation_ft                   358082 non-null float64
29   le_heading_degT                   358082 non-null float64
30   le_displaced_threshold_ft         83200 non-null float64
31   he_ident                          358082 non-null object
32   he_latitude_deg                   358082 non-null float64
33   he_longitude_deg                  358082 non-null float64
34   he_elevation_ft                   352381 non-null float64
35   he_heading_degT                   358082 non-null float64
36   he_displaced_threshold_ft         57704 non-null float64
37   Founded                           358082 non-null float64
38   Rank (2024)                       274167 non-null float64
39   Airports (large)                  274167 non-null object
40   Major cities served               274167 non-null object
41   Metro area                        274167 non-null object
42   State                             358082 non-null object
43   2023[3]                           358082 non-null int64
44   2022[4]                           358082 non-null int64
45   2019[7]                           358082 non-null int64
46   2018[8]                           358082 non-null int64
47   2017[9]                           358082 non-null int64
48   2016[10]                          358082 non-null int64
```

```
In [68]: # Now let's check the null value and treat them.
         final_df.isnull().sum()
```

```
Out[68]: Airline                0
         Flight                0
         DayOfWeek             0
         Time                  0
         Length                0
         Delay                 0
         Type                  0
         latitude_deg           0
         longitude_deg          0
         elevation_ft           0
         continent              352185
         iso_country            0
         iso_region             0
         municipality           0
         scheduled_service      0
         gps_code               0
         local_code             0
         home_link              64988
         wikipedia_link         2086
         keywords               146949
         airport_ref            0
         length_ft              0
         width_ft               0
         lighted                0
         closed                 0
         le_ident               0
         le_latitude_deg        0
         le_longitude_deg       0
         le_elevation_ft        0
         le_heading_degT        0
         le_displaced_threshold_ft 274793
         he_ident               0
         he_latitude_deg        0
         he_longitude_deg       0
         he_elevation_ft        2699
         he_heading_degT        0
         he_displaced_threshold_ft 300238
         Founded                0
         Rank (2024)            83835
         Airports (large)       83835
         Major cities served     83835
         Metro area              83835
         State                   0
         2023[3]                 0
         2022[4]                 0
         2019[7]                 0
         2018[8]                 0
         2017[9]                 0
         2016[10]                0
         2015[11]                0
         traffic_chg18_19       83835
         traffic_chg17_18       83835
         hubs                    83835
```



```
In [61]: final_df
```

```
Out[61]:
```

	Airline	Flight	DayOfWeek	Time	Length	Delay		type	latitude_deg	longitude_deg	elevation_ft	...	traffic_Cat16_19	traffic_Cat17_18	n
0	AA	2468	3	20	195	1	large_airport	37.619900	-122.375000	13.0	...	11457.0	1152947.0	large	
1	AA	520	3	200	215	0	large_airport	37.619999	-122.375000	13.0	...	11457.0	1152947.0	large	
2	AA	302	3	209	315	1	large_airport	37.619999	-122.375000	13.0	...	11457.0	1152947.0	large	
3	AA	810	3	265	255	0	large_airport	37.619999	-122.375000	13.0	...	11457.0	1152947.0	large	
4	AA	24	3	125	325	1	large_airport	37.619999	-122.375000	13.0	...	11457.0	1152947.0	large	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
367207	9E	4052	4	370	90	0	medium_airport	32.847099	-90.361799	487.0	...	NaN	NaN		
367208	9C	3704	4	704	82	1	medium_airport	32.847099	-90.361799	487.0	...	NaN	NaN		
367209	9C	4060	4	990	90	0	medium_airport	32.847099	-90.361799	487.0	...	NaN	NaN		
368000	9E	4052	5	370	90	1	medium_airport	32.847099	-90.361799	487.0	...	NaN	NaN		
368001	9C	3704	5	704	82	0	medium_airport	32.847099	-90.361799	487.0	...	NaN	NaN		

350802 rows x 16 columns

```
In [62]: final_df.dropna(axis=0, inplace=True)
```

```
In [92]: print(df.shape)
print(df.head())
```

```
(434924, 47)
   id  airline  Flight  AirportFrom  AirportTo  DayOfWeek  Time  Length  Delay \
0    4      AA    2468          SFO          DFW          3     20     195    1
1   231      AA    520          SFO          DFW          3    300     215    0
2   234      AA    152          SFO          MIA          3    360     215    1
3   905      AA    810          SFO          ORD          3    305     255    0
4  1739      AA     24          SFO          JFK          3    425     315    1

   ident  ...  le_heading_degT  le_displaced_threshold_ft  he_ident \
0  1570  ...      28.0          491.0          195
1  1570  ...      28.0          491.0          195
2  1570  ...      28.0          491.0          195
3  1570  ...      28.0          491.0          195
4  1570  ...      28.0          491.0          195

   he_latitude_deg  he_longitude_deg  he_elevation_ft  he_heading_degT \
0    37.627391      -122.375000          0.0          208.0
1    37.627391      -122.375000          0.0          208.0
2    37.627391      -122.375000          0.0          208.0
```

```
In [93]: print(df['Delay'].value_counts(dropna=False))
```

```
0    135745
1     90970
Name: Delay, dtype: int64
```

```
In [94]: plt.figure(figsize=(10,7))
sns.countplot(data=final_df, x='Airline', hue='Delay')
plt.xticks(rotation=45) # Optional: rotate x-axis labels for readability
plt.title('Flight Delays by Airline')
plt.xlabel('Airline')
plt.ylabel('Number of Flights')
plt.tight_layout()
plt.show()
```

```
ValueError                                Traceback (most recent call last)
/tmpipykernel_58/1428573167.py in <cell line: 2>()
      1 plt.figure(figsize=(10,7))
----> 2 sns.countplot(data=final_df, x='Airline', hue='Delay')
      3 plt.xticks(rotation=45) # Optional: rotate x-axis labels for readability
      4 plt.title('Flight Delays by Airline')
      5 plt.xlabel('Airline')

~/usr/local/lib/python3.10/site-packages/seaborn/categorical.py in countplot(data, x, y, hue, order, hue_order, orient, color, p
alette, saturation, width, dodge, ax, **kwargs)
    2941     raise ValueError("Cannot pass values for both 'x' and 'y'")
    2942
-> 2943     plotter = CountPlotter(
    2944         x, y, hue, data, order, hue_order,
    2945         estimator, errorbar, n_boot, units, seed,

~/usr/local/lib/python3.10/site-packages/seaborn/categorical.py in _init_(self, x, y, hue, data, order, hue_order, estimator,
errorbar, n_boot, units, seed, orient, color, palette, saturation, width, errorcolor, errorwidth, capsize, dodge)
    1530     self.establish_variables(x, y, hue, data, orient,
    1531                             order, hue_order, units)
-> 1532     self.establish_colors(color, palette, saturation)
    1533     self.estimate_statistic(estimator, errorbar, n_boot, seed)
    1534

~/usr/local/lib/python3.10/site-packages/seaborn/categorical.py in establish_colors(self, color, palette, saturation)
    765     # Determine the gray color to use for the lines framing the plot
    766     light_val = [rgb_to_hls(c)[2] for c in rgb_colors]
-> 767     lum = min(light_val) + 0
    768     gray = mpl.colors.rgb2hsv((lum, lum, lum))
    769
```

```
ValueError: min() arg is an empty sequence
```

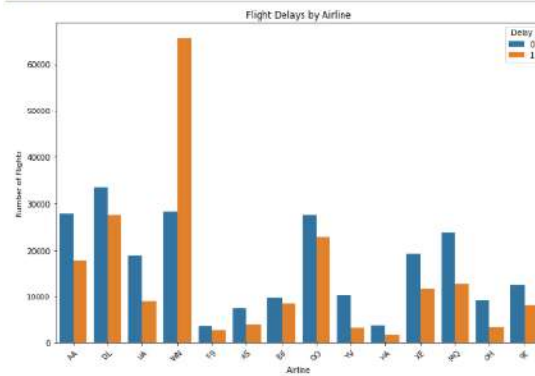
```
<Figure size 720x504 with 0 Axes>
```

```
In [95]: print(final_df.shape)
```

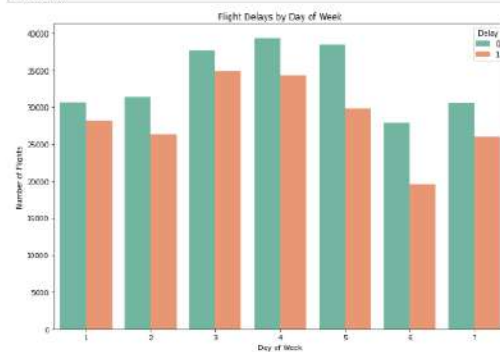
```
(0, 68)
```

```
In [96]: final_df = df[df['Delay'].isin([0, 1]) & df['Airline'].notnull()]
```

```
In [97]: plt.figure(figsize=(10,7))
sns.countplot(data=final_df, x='Airline', hue='Delay')
plt.xticks(rotation=45)
plt.title('Flight Delays by Airline')
plt.xlabel('Airline')
plt.ylabel('Number of Flights')
plt.tight_layout()
plt.show()
```



```
In [99]: plt.figure(figsize=(10,7))
sns.countplot(data=final_df, x='DayOfWeek', hue='Delay', palette='Set2')
plt.title('Flight Delays by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Number of Flights')
plt.tight_layout()
plt.show()
```

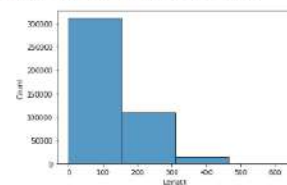


```
In [100]: final_df['length'].max()
```

```
Out[100]: 520
```

```
In [101]: sns.histplot(final_df['length'], bins = 4)
```

```
Out[101]: <AxesSubplot: xlabel='length', ylabel='Count'>
```



```
In [102]: final_df['Airline'][(final_df['Length']<180)].value_counts()

Out[102]: WN      80256
          OO      48335
          DL      43010
          NQ      35405
          XE      29962
          AA      20523
          SE      20022
          UA      10044
          YV      13419
          OH      12149
          BG      11857
          AS      7791
          F9      5251
          HA      4575
          Name: Airline, dtype: int64

In [104]: final_df['Airline'][(final_df['Length']>180) & (final_df['Length']<360)].value_counts()

Out[104]: DL      16869
          AA      16559
          WN      12509
          UA      9576
          BG      5340
          AS      3021
          OO      1842
          F9      1205
          XE      1149
          NQ      770
          HA      751
          OH      434
          YV      306
          SE      64
          Name: Airline, dtype: int64

In [105]: final_df['Airline'][(final_df['Length']>360)].value_counts()

Out[105]: UA      1314
          AA      1081
          DL      842
          BG      822
          AS      540
          HA      252
          WN      139
          Name: Airline, dtype: int64

In [106]: final_df[final_df['Length']>360].describe().T
```

```
Out[106]:
```

		count	mean	std	min	25%	50%	75%	max
	id	4990.0	272048.851303	155498.802520	24.000000	137703.000000	273651.500000	406337.750000	539373.000000
	Flight	4990.0	558.466733	748.411134	1.000000	64.000000	209.000000	849.000000	3760.000000
	DayOfWeek	4990.0	4.005210	1.926087	1.000000	2.000000	4.000000	6.000000	7.000000
	Time	4990.0	826.509018	284.686593	100.000000	550.000000	870.000000	1079.000000	1435.000000
	Length	4990.0	395.405611	40.501390	361.000000	374.000000	385.000000	400.000000	620.000000
	Delay	4990.0	0.414028	0.492603	0.000000	0.000000	0.000000	1.000000	1.000000
	latitude_deg	4990.0	38.965243	6.500728	18.439420	38.944500	40.639447	41.978600	61.174400
	longitude_deg	4990.0	-88.130077	25.194429	-159.339005	-97.038002	-73.779317	-73.779317	-66.001801
	elevation_ft	4990.0	236.362725	740.073240	8.000000	13.000000	18.000000	125.000000	5431.000000
	airport_ref	4990.0	3745.469739	490.270415	3384.000000	3602.000000	3622.000000	3670.000000	6384.000000
	length_ft	4990.0	10017.873148	2104.171108	4892.000000	7861.000000	10600.000000	12079.000000	12079.000000
	width_ft	4990.0	171.242485	25.000688	100.000000	150.000000	150.000000	200.000000	200.000000
	lighted	4990.0	0.952305	0.213142	0.000000	1.000000	1.000000	1.000000	1.000000
	closed	4990.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
	le_latitude_deg	4990.0	38.947879	6.486209	18.438801	38.936098	40.622002	41.981701	61.169800
	le_longitude_deg	4990.0	-88.136799	25.194864	-159.349999	-97.021500	-73.785599	-73.785599	-66.014500
	le_elevation_ft	4990.0	228.605010	732.167934	6.000000	12.000000	12.000000	112.000000	5347.000000
	le_heading_degT	4990.0	52.440100	49.284991	1.000000	31.000000	31.000000	53.000000	359.000000
	le_displaced_threshold_ft	406.0	1390.788177	925.844731	577.000000	587.000000	682.500000	2543.000000	2543.000000
	he_latitude_deg	4990.0	38.963809	6.500021	18.449499	38.970600	40.648800	41.997501	61.169800
	he_longitude_deg	4990.0	-88.119781	25.194331	-159.335007	-97.000801	-73.764702	-73.764702	-65.968297
	he_elevation_ft	4990.0	221.686172	726.965211	5.000000	13.000000	13.000000	117.000000	5352.000000
	he_heading_degT	4990.0	230.997214	47.035086	5.000000	211.000000	211.000000	233.000000	360.000000
	he_displaced_threshold_ft	2993.0	2105.322419	852.494784	205.000000	819.000000	2696.000000	2696.000000	2696.000000
	Founded	4990.0	1939.465932	26.918608	1924.000000	1926.000000	1926.000000	1932.000000	1998.000000

```
In [107]: sns.countplot(final_df['hubs'], hue = final_df['Delay'])
```

```
In [107]: sns.countplot(final_df['hubs'], hue = final_df['Delay'])
```

```

KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.10/site-packages/pandas/core/indexes/base.py in get_loc(self, key, method, tolerance)
   3801     try:
-> 3802         return self._engine.get_loc(casted_key)
   3803     except KeyError as err:
        raise

/usr/local/lib/python3.10/site-packages/pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()
/usr/local/lib/python3.10/site-packages/pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()
pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()
pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'hubs'

The above exception was the direct cause of the following exception:

KeyError                                Traceback (most recent call last)
/tmp/ipykernel_68/232549111.py in <cell line: 1>()
----> 1 sns.countplot(final_df['hubs'], hue = final_df['Delay'])

/usr/local/lib/python3.10/site-packages/pandas/core/frame.py in __getitem__(self, key)
   3805     if self.columns.nlevels > 1:
   3806         return self._getitem_multilevel(key)
-> 3807     indexer = self.columns.get_loc(key)
   3808     if is_integer(indexer):
   3809         indexer = [indexer]

/usr/local/lib/python3.10/site-packages/pandas/core/indexes/base.py in get_loc(self, key, method, tolerance)
   3802     return self._engine.get_loc(casted_key)
   3803     except KeyError as err:
-> 3804         raise KeyError(key) from err
   3805     except TypeError:
   3806         # If we have a listlike key, _check_indexing_error will raise
        raise

KeyError: 'hubs'

```

```
In [108]: print(final_df.columns.tolist())
```

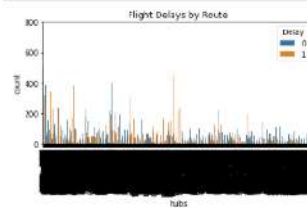
```

['id', 'Airline', 'Flight', 'AirportFrom', 'AirportTo', 'DayOfWeek', 'Time', 'Length', 'Delay', 'ident', 'type', 'name', 'latitude_deg', 'longitude_deg', 'elevation_ft', 'continent', 'iso_country', 'iso_region', 'municipality', 'scheduled_service', 'gps_code', 'iata_code', 'local_code', 'home_link', 'wikipedia_link', 'keywords', 'airport_ref', 'airport_ident', 'length_ft', 'width_ft', 'surface', 'lighted', 'closed', 'le_ident', 'le_latitude_deg', 'le_longitude_deg', 'le_elevation_ft', 'le_heading_degT', 'le_displaced_threshold_ft', 'he_ident', 'he_latitude_deg', 'he_longitude_deg', 'he_elevation_ft', 'he_heading_degT', 'he_displaced_threshold_ft', 'IATA', 'Founded']

```

```
In [109]: final_df['hubs'] = final_df['AirportFrom'] + ' + ' + final_df['AirportTo']
```

```
In [110]: sns.countplot(data=final_df, x='hubs', hue='Delay')
plt.xticks(rotation=90)
plt.title('Flight Delays by Route')
plt.tight_layout()
plt.show()
```



```
In [113]: from scipy.stats import chi2_contingency
table = [final_df['latitude_deg'], final_df['Delay']]
stat, p, dof, expected = chi2_contingency(table)
print('stat=%.3f, p=%.3f' % (stat, p))
if p > 0.05: print('Probably independent')
else: print('Probably dependent')

stat=140862.542, p=1.000
Probably independent
```

```
In [114]: from scipy.stats import chi2_contingency
table = [final_df['airport_ref'], final_df['Delay']]
stat, p, dof, expected = chi2_contingency(table)
print('stat=%.3f, p=%.3f' % (stat, p))
if p > 0.05: print('Probably independent')
else: print('Probably dependent')

stat=107195.810, p=1.000
Probably independent
```

```
In [115]: from scipy.stats import spearmanr
data1 = final_df['length']
data2 = final_df['Delay']
stat, p = spearmanr(data1, data2)
print('stat=%.3f, p=%.3f' % (stat, p))
if p > 0.05: print('Probably independent')
else: print('Probably dependent')
```

Heatmap showing the relative abundance of 16S rDNA sequences across 16 different samples. The color scale ranges from 0.0 (blue) to 1.0 (red). The samples are labeled on the y-axis: 1a, 1b, 1c, 1d, 1e, 1f, 1g, 1h, 1i, 1j, 1k, 1l, 1m, 1n, 1o, 1p. The x-axis labels are: 1a, 1b, 1c, 1d, 1e, 1f, 1g, 1h, 1i, 1j, 1k, 1l, 1m, 1n, 1o, 1p. The heatmap shows a high degree of similarity between samples, with most sequences having a relative abundance of 1.0 (red). Some sequences show lower relative abundance (blue/green) in specific samples, such as 1a, 1b, 1c, 1d, 1e, 1f, 1g, 1h, 1i, 1j, 1k, 1l, 1m, 1n, 1o, 1p.

```
In [124]: final_df['AirLine'].value_counts()
Out[124]:
Air  94897
DL   68048
OO   58314
AA   45556
MQ   36774
B6   31116
UA   27929
9E   26566
B3   18812
VY   13215
OH   12510
AS   11471
F4   6416
HA   5518
Name: AirLine, dtype: int64

In [125]: final_df['Type'].value_counts()
Out[125]:
large_airport    351588
medium_airport   82218
small_airport     1186
Name: Type, dtype: int64

In [126]: final_df['scheduled_service'].value_counts()
Out[126]:
yes      43253
no       1978
Name: scheduled_service, dtype: int64

In [127]: final_df['Hub'].value_counts()
Out[127]:
LAX = SFO    1879
SFO = LAX    1877
OGG = HNL    982
HNL = GEG    521
SAN = LAX    335
...
SFO = LAX    1
PVD = PSP    1
IDA = FDI    1
DFW = LBS    1
LAX = HOU    1
Name: Hub, length: 3782, dtype: int64

In [128]: final_df = final_df.drop(['scheduled_service'], axis=1)

In [129]: # Now using the ordinal encoder
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

In [130]: final_df['AirLine'] = le.fit_transform(final_df['AirLine'])
final_df['Type'] = le.fit_transform(final_df['Type'])
```



```
In [133]: from sklearn import preprocessing
scaler = preprocessing.MinMaxScaler()
x = scaler.fit_transform(x)

ValueError                                Traceback (most recent call last)
~/tmp/ipykernel_68/388305116.py in cell line 30()
      1 from sklearn import preprocessing
      2 scaler = preprocessing.MinMaxScaler()
----> 3 x = scaler.fit_transform(x)

~/usr/lib/python3.10/site-packages/sklearn/utils/_set_output.py in wrapper(self, X, *args, **kwargs)
   155     @wraps(f)
   156     def wrapped(self, X, *args, **kwargs):
--> 157         data_to_wrap = f(self, X, *args, **kwargs)
   158         if isinstance(data_to_wrap, list):
   159             # Only wrap the first output for cross decomposition
   160             # Only wrap the first output for cross decomposition

~/usr/lib/python3.10/site-packages/sklearn/base.py in fit_transform(self, X, y, **fit_params)
   954     fit y in X now:
   925     # Fit method or arity 1 (unsupervised transformation)
--> 926     return self.fit(X, **fit_params).transform(X)
   927     else:
   928         # Fit method or arity 2 (supervised transformation)

~/usr/lib/python3.10/site-packages/sklearn/preprocessing/_data.py in fit(self, X, y)
   412     # Reset internal state before fitting
   434     self._reset()
--> 435     return self.partial_fit(X, y)
   436
   437     @fit_context(prefer_skip_nested_validation=True)

~/usr/lib/python3.10/site-packages/sklearn/base.py in wrapper(estimator, *args, **kwargs)
   1150     (
   1151         )
```

```
In [135]: from sklearn.preprocessing import LabelEncoder

label_encoders = {}

for col in categorical_cols:
    le = LabelEncoder()
    x[col] = le.fit_transform(x[col])
    label_encoders[col] = le # Save encoder if you need to inverse later

TypeError                                 Traceback (most recent call last)
~/usr/local/lib/python3.10/site-packages/sklearn/utils/_encode.py in _unique_python(values, return_inverse, return_counts)
    173     uniques = sorted(unique_set)
--> 174     uniques.extend(missing_values.to_list())
    175

TypeError: 'c' not supported between instances of 'int' and 'str'

During handling of the above exception, another exception occurred:
```

```

4 for col in categorical_cols:
5     le = LabelEncoder()
----> 6     x[col] = le.fit_transform(x[col])
7     label_encoders[col] = le # Save encoder if you need to inverse later

/usr/local/lib/python3.10/site-packages/sklearn/preprocessing/_label.py in fit_transform(self, y)
113     ...
114     y = column_or_1d(y, warn=True)
--> 115     self.classes_, y = _unique(y, return_inverse=True)
116     return y

/usr/local/lib/python3.10/site-packages/sklearn/utils/_encode.py in _unique(values, return_inverse, return_counts)
40     ...
41     if values.dtype == object:
--> 42         return _unique_python(
43             values, return_inverse=return_inverse, return_counts=return_counts)
44     )

/usr/local/lib/python3.10/site-packages/sklearn/utils/_encode.py in _unique_python(values, return_inverse, return_counts)
177     except TypeError:
178         types = sorted(t.__qualname__ for t in set(type(v) for v in values))
--> 179         raise TypeError(
180             "Encoders require their input argument must be uniformly "
181             f"{'strings or numbers.' if types == ['str']}")

TypeError: Encoder's require their input argument must be uniformly strings or numbers. Got ['int', 'str']

In [130]: from sklearn.preprocessing import LabelEncoder
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    x[col] = x[col].astype(str) # Convert all values to string
    x[col] = le.fit_transform(x[col])
    label_encoders[col] = le

In [137]: from sklearn import preprocessing
scaler = preprocessing.MinMaxScaler()
x = scaler.fit_transform(x)

In [139]: # First Split the data into the training and testing set before performing the further operation.
from sklearn.model_selection import train_test_split

In [141]: x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.9, random_state=49)

In [142]: # Lets apply the Logistic regression with the randomsearchcv hypermeter tuning.
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()

In [143]: from sklearn.model_selection import RandomizedSearchCV

```

```

In [144]: param = {'penalty': ['l1', 'l2'], 'solver': ['newton-cg', 'liblinear']}
# Cross Validation
folds = 5
rscv = RandomizedSearchCV(estimator = lr, param_distributions = param, scoring = "accuracy", verbose = 1, cv= folds)

In [145]: rscv.fit(x_train, y_train)

1150         )
1151     ):
--> 1152         return fit_method(estimator, *args, **kwargs)
1153     return wrapper

/usr/local/lib/python3.10/site-packages/sklearn/model_selection/_search.py in fit(self, X, y, groups, **fit_params)
896     return results
897
--> 898     self._run_search(evaluate_candidates)
899
900     # multi-metric is determined here because in the case of a callable
901
/usr/local/lib/python3.10/site-packages/sklearn/model_selection/_search.py in _run_search(self, evaluate_candidates)
1807     def _run_search(self, evaluate_candidates):
1808         """Search n_iter candidates from param_distributions"""
--> 1809         evaluate_candidates(
1810             ParameterSampler(
1811                 self.param_distributions, self.n_iter, random_state=self.random_state

```

```

In [146]: print(x_train.shape, y_train.shape)
print(x_train.dtype)
print(x_train.tolist().sum())

(101431, 45) (101431,)

-----
AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_98/3054089160.py in <cell line: 2>()
1 print(x_train.shape, y_train.shape)
----> 2 print(x_train.dtype)
3 print(x_train.tolist().sum())

AttributeError: 'numpy.ndarray' object has no attribute 'dtype'

In [147]: from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
X_train = scaler.fit_transform(x_train)

In [ ]: rscv.fit(x_train, y_train)

In [151]: x_train_scaled = scaler.fit_transform(x_train)
X_train_of = pd.DataFrame(x_train_scaled, columns=predictor.columns)

-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_98/3266001600.py in <cell line: 2>()
1 x_train_scaled = scaler.fit_transform(x_train)

```

```

ValueError                                Traceback (most recent call last)
/tmp/ipykernel_68/3264003895.py in <cell line: 2>()
    1 x_train_scaled = scaler.fit_transform(x_train)
----> 2 x_train_df = pd.DataFrame(x_train_scaled, columns=predictor.columns)

/usr/local/lib/python3.10/site-packages/pandas/core/frame.py in __init__(self, data, index, columns, dtype, copy)
    710     )
    711     else:
--> 712         mgr = ndarray_mgr(
    713             data,
    714             index,

/usr/local/lib/python3.10/site-packages/pandas/core/internals/construction.py in ndarray_mgr(values, index, columns, dtype, copy, typ)
    347     )
    348     _check_values_indices_shape_match(values, index, columns)
--> 349     _check_values_indices_shape_match(values, index, columns)
    350     if typ == "array":
    351         if typ == "array":

/usr/local/lib/python3.10/site-packages/pandas/core/internals/construction.py in _check_values_indices_shape_match(values, index, columns)
    418     passed = values.shape
    419     implied = (len(index), len(columns))
--> 420     raise ValueError(f"Shape of passed values is {passed}, indices imply {implied}")
    421
    422

ValueError: Shape of passed values is (391431, 46), indices imply (391431, 47)

In [152]: print(predictor.shape)
          print(predictor.columns)

(434824, 47)
Index([ 10, 'Airline', 'flight', 'airportfrom', 'airportto', 'dayofweek',
       'time', 'length', 'ident', 'type', 'name', 'latitude_deg',
       'longitude_deg', 'elevation_ft', 'continent', 'iso_country',
       'iso_region', 'municipality', 'scheduled_service', 'gso_code',
       'iata_code', 'local_code', 'home_link', 'wikipedia_link', 'keywords',
       'airport_ref', 'airport_ident', 'length_ft', 'width_ft', 'surface',
       'lighted', 'closed', 'is_ident', 'is_latitude_deg', 'is_longitude_deg',
       'is_elevation_ft', 'is_heading_deg', 'is_displaced_threshold_ft',
       'is_ident', 'is_latitude_deg', 'is_longitude_deg', 'is_elevation_ft',
       'is_heading_deg', 'is_displaced_threshold_ft', 'iata', 'founded',
       'thumbs'],
      dtype=object)

In [153]: rscv.fit(x_train, y_train)

/usr/local/lib/python3.10/site-packages/sklearn/model_selection/_search.py:387: UserWarning: The total space of parameters
4 is smaller than n_iter=10. Running 4 iterations. For exhaustive searches, use GridSearchCV.
searching space

Fitting 5 folds for each of 4 candidates, totalling 20 fits

ValueError                                Traceback (most recent call last)
/tmp/ipykernel_68/3469245214.py in <cell line: 3>()
----> 1 rscv.fit(x_train, y_train)

/usr/local/lib/python3.10/site-packages/sklearn/model_selection/_search.py:387: UserWarning: The total space of parameters
4 is smaller than n_iter=10. Running 4 iterations. For exhaustive searches, use GridSearchCV.
searching space

Fitting 5 folds for each of 4 candidates, totalling 20 fits

```

```

Out[154]: RandomizedSearchCV
          estimator: LogisticRegression
          estimator__ LogisticRegression

In [155]: print(rscv.best_params_)
          print(rscv.best_score_)

AttributeError                                Traceback (most recent call last)
/tmp/ipykernel_68/3469245214.py in <cell line: 3>()
----> 1 print(rscv.best_params_)
      2 print(rscv.best_score_)

AttributeError: "RandomizedSearchCV" object has no attribute 'best_params_'

In [156]: try:
          print('Best Parameters:', rscv.best_params_)
          print('Best Score:', rscv.best_score_)
except AttributeError:
    print('Oops! Looks like you need to fit the model first using rscv.fit(x_train, y_train)')
    print('Oops! Looks like you need to fit the model first using rscv.fit(x_train, y_train)')

In [159]: from sklearn.input_data import SimpleInputer
          from sklearn.linear_model import LogisticRegression
          from sklearn.pipeline import Pipeline

          # Create a pipeline that inputs and then fits the model
          pipeline = Pipeline([
              ('inputer', SimpleInputer(strategy='none')), # or 'median', 'most_frequent'
              ('logreg', LogisticRegression(penalty='l2', solver='newton-g'))
          ])

          pipeline.fit(x_train, y_train)
          score = pipeline.score(x_train, y_train)
          print("Training score:", score)

          Training score: 0.617688136721161

In [173]: from sklearn.tree import DecisionTreeClassifier
          dt = DecisionTreeClassifier()

          params = {'criterion': ['gini', 'entropy'], 'min_samples_leaf': [2,3,4,5,6,7,8,9], 'max_depth': [2,3,4,5,6,7,8,9]}
          rscv = RandomizedSearchCV(estimator = dt, param_distributions = params, scoring = 'accuracy', cv= 5, verbose=1)
          rscv.fit(x_train, y_train)

          Fitting 5 folds for each of 20 candidates, totalling 50 fits

Out[173]: RandomizedSearchCV
          estimator: DecisionTreeClassifier
          estimator__ DecisionTreeClassifier

```

```
In [175]: print(rscv.best_params_)
print(rscv.best_score_)
{'min_samples_leaf': 4, 'max_depth': 6, 'criterion': 'gini'}
0.648003273179136

In [177]: dtc = DecisionTreeClassifier(max_depth=9, criterion='entropy', min_samples_leaf=6)
dtc.fit(x_train, y_train).score(x_train, y_train)

Out[177]: 0.6689562562881785

In [178]: dtc.score(x_test, y_test)

Out[178]: 0.668568205808878

In [ ]: from xgboost import XGBClassifier

# Create the parameter grid: gbm_param_grid
gbm_param_grid = {'n_estimators': range(8, 20), 'max_depth': range(6, 10), 'learning_rate': [.4, .45, .5, .55, .6], 'colsample_bytree': 0.5}

# Instantiate the regressor: gbm
gbm = XGBClassifier()

# Perform random search: grid_mse
xgb_random = RandomizedSearchCV(param_distributions=gbm_param_grid, estimator=gbm, scoring="accuracy", verbose=1, n_iter=5)

# Fit randomized mse to the data
xgb_random.fit(x_train, y_train)

# Print the best parameters and lowest RMSE
print("Best parameters found: ", xgb_random.best_params_)
print("Best accuracy found: ", xgb_random.best_score_)

In [182]: try:
    xgb_random.fit(x_train, y_train)
    print("Best parameters found: ", xgb_random.best_params_)
    print("Best accuracy found: ", xgb_random.best_score_)
except Exception as e:
    print("Error during fitting:", e)

Fitting 3 folds for each of 50 candidates, totalling 150 fits
Best parameters found: {'n_estimators': 18, 'max_depth': 9, 'learning_rate': 0.5, 'colsample_bytree': 0.6}
Best accuracy found: 0.698764338728094

In [ ]: from xgboost import XGBClassifier
from sklearn.model_selection import RandomizedSearchCV

# Define the parameter grid
gbm_param_grid = {
    'n_estimators': range(8, 20),
    'max_depth': range(6, 10)
}
```

```
In [ ]: from xgboost import XGBClassifier
from sklearn.model_selection import RandomizedSearchCV

# Define the parameter grid
gbm_param_grid = {
    'n_estimators': range(8, 20),
    'max_depth': range(6, 10),
    'learning_rate': [0.4, 0.45, 0.5, 0.55, 0.6],
    'colsample_bytree': [0.6, 0.7, 0.8, 0.9, 1.0]
}

# Instantiate the classifier
gbm = XGBClassifier(use_label_encoder=False, eval_metric='logloss')

# Set up RandomizedSearchCV
xgb_random = RandomizedSearchCV(
    estimator=gbm,
    param_distributions=gbm_param_grid,
    scoring='accuracy',
    verbose=1,
    n_iter=50,
    cv=5
)

# Fit the model
xgb_random.fit(x_train, y_train)

# Print the best parameters and best accuracy score
print("Best parameters found: ", xgb_random.best_params_)
print("Best accuracy score: ", xgb_random.best_score_)

In [186]: xgb = XGBClassifier(n_estimators=14, max_depth=9, learning_rate=0.45, colsample_bytree=0.9)
xgb.fit(x_train, y_train).score(x_train, y_train)

Out[186]: 0.726626063723622

In [ ]: # Now lets compare the all method.
print(lr.score(x_test, y_test))
print(dtc.score(x_test, y_test))
print(xgb.score(x_test, y_test))

In [12]: from sklearn.datasets import load_iris

# Load sample data
data = load_iris()
X = data.data
y = data.target

# Split it
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

In [ ]:
```